



Introduction to Web Technologies

**SWE230 :
WEB APPLICATION PROGRAMMING**

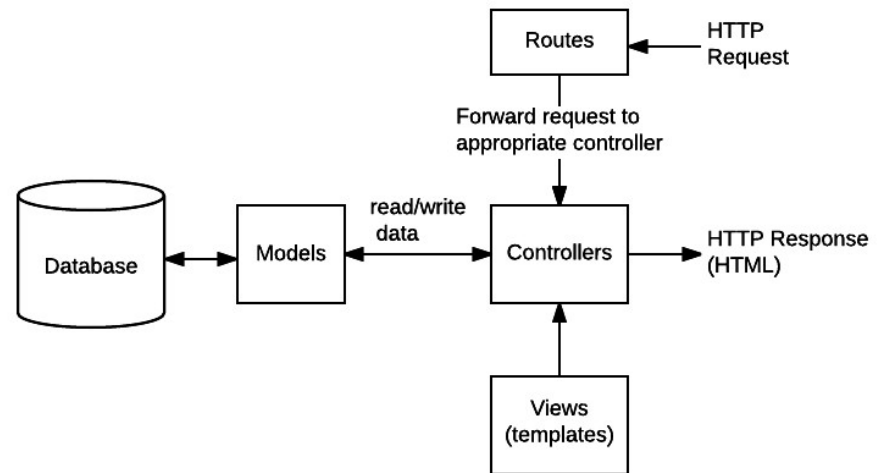
Spring 2026

Instructors

- **Sarah Nabil**
Sara.abdullah@miuegypt.edu.eg
- **Nada Ayman**
nada.ayman@miuegypt.edu.eg
- **Nada AbdelFattah**
nada.abdelfattah@miuegypt.edu.eg

What is MVC Architecture?

- MVC stands for Model-View-Controller.
 - Model: Handles data and database logic.
 - View: The user interface (EJS templates).
 - Controller: Business logic and request handling.
- Helps organize code and separate concerns.



MVC Architecture

Project Structure

- project/
 - controllers/
 - models/
 - views/
 - routes/
 - public/
 - css/
 - js/
 - images/

Web client examples

What is Curl and why use it?

- Curl is a command-line tool that allows us to do HTTP requests from shell.
- Curl is very useful for testing the output of an API before deploying it to production, or simply fetching a response from a website (for instance, to check it's not down), Curl is practically predominant.
- Its name stands for "Client URL", and it was developed by Swedish developer Daniel Stenberg. It is an open source project, check:

<https://github.com/curl/curl>

- Install steps on windows:
<https://linuxhint.com/install-use-curl-windows/>



Calling a GET method

- In its most basic form, a curl command will look like this:

```
curl http://www.dataden.tech
```

- To direct the response's contents into a file. This is done with the `-o` argument:

```
curl -o output.html www.dataden.tech
```

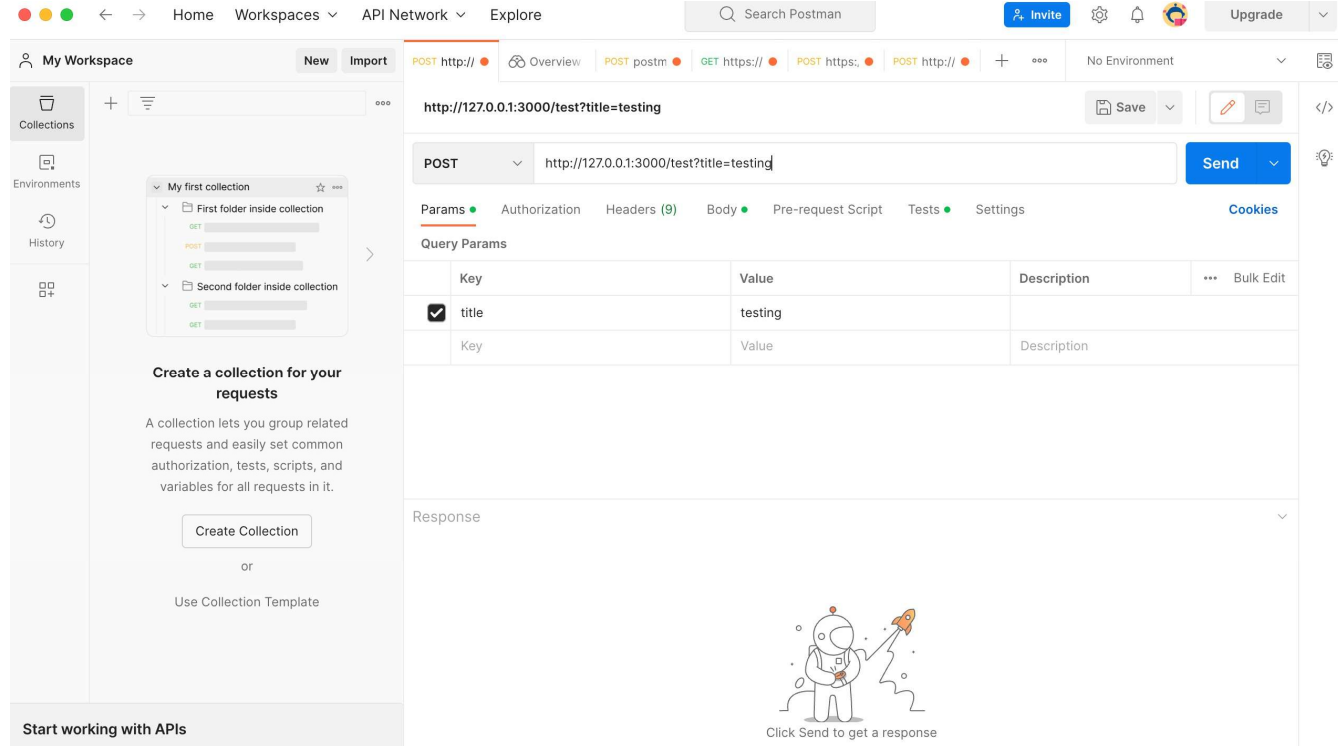
Using the POST method

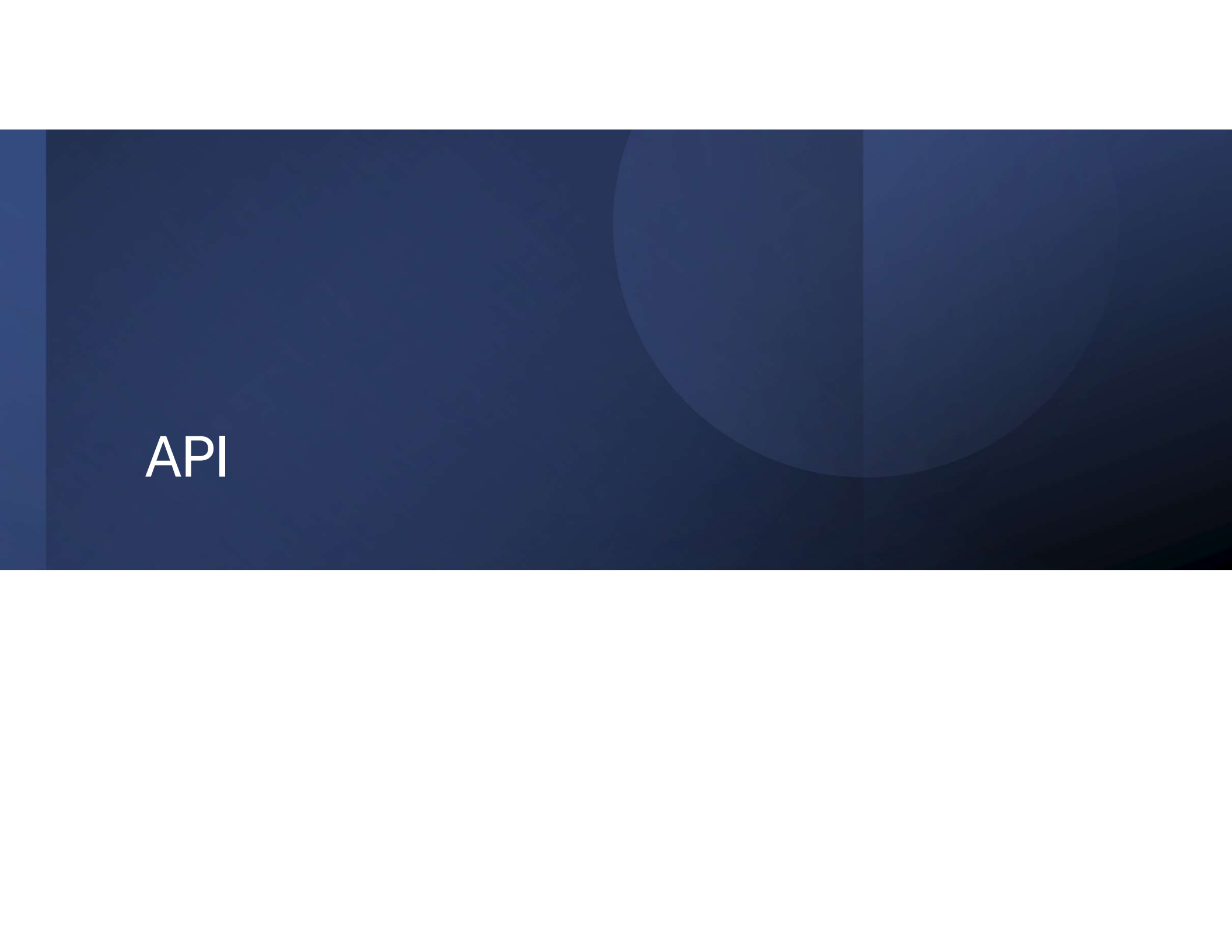
- To pass data (Usually in JSON format) with all the required parameters. There are many ways to do this with curl.

curl --data "name=John&surname=Doe" <http://www.dataden.tech>

Postman

- <https://www.postman.com/downloads/>

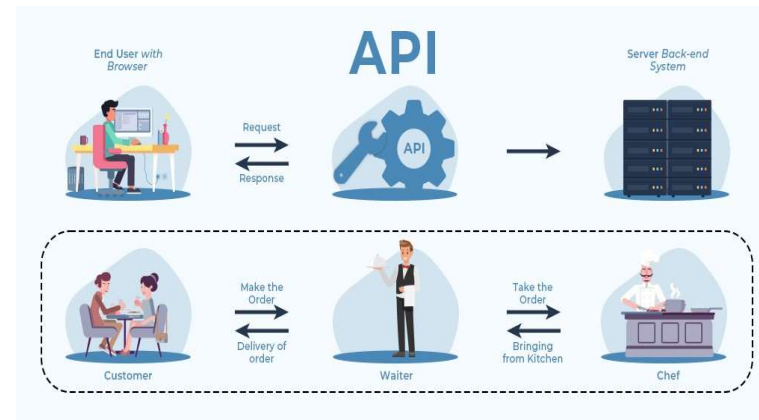




API

API

- API stands for Application Programming Interface.
- In the context of APIs, the word Application refers to any software with a distinct function.
- Interface can be thought of as a contract of service between two applications. This contract defines how the two communicate with each other using requests and responses.

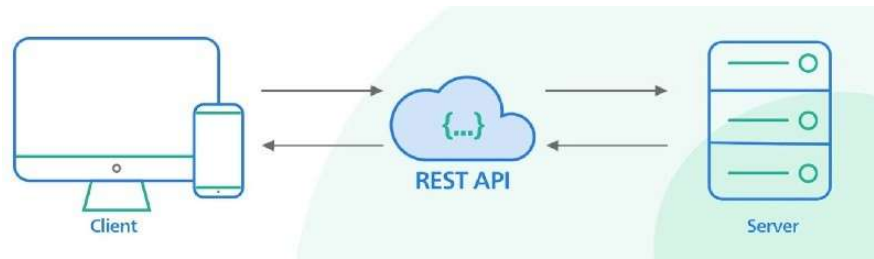


How do APIs work?

- API architecture is usually explained in terms of client and server. The application sending the request is called the client, and the application sending the response is called the server.
- There are four different ways that APIs can work:

- **SOAP APIs**
- **RPC APIs**
- **Websocket APIs**
- **REST APIs**

- (the most popular and flexible APIs found on the web today)



REST (REpresentational State Transfer)

- REST stands for Representational State Transfer. REST defines a set of functions like GET, POST, PUT, DELETE, PATCH. that clients can use to access server data. Clients and servers exchange data using HTTP.
- The main feature of [REST API](#) is **statelessness**. Statelessness means that servers do not save client data between requests.



What are API integrations?

- API integrations are software components that automatically update data between clients and servers.
- Some examples of API integrations are when automatic data sync to the cloud from your phone image gallery, or the time and date automatically sync on your laptop.
- Enterprises can use API integrations to automate many system functions.

Benefits of REST APIs

1. **Integration** increases development speed because each functionality doesn't have to be written from scratch. You can use APIs to leverage existing code.
2. **Innovation** Businesses need to respond quickly and support the rapid deployment of innovative services. They can do this by making changes at the API level without having to re-write the whole code.
3. **Expansion** Any business can give access to their internal databases by using free or paid APIs.
4. **Ease of maintenance** The API acts as a gateway between two systems. Each system is obliged to make internal changes so that the API is not impacted. This way, any future code changes by one party do not impact the other party.

Different types of APIs (based on scope of use)

- **Private APIs** These are internal to an enterprise and only used for connecting systems and data within the business.
- **Public APIs** These are open to the public and may be used by anyone. There may or not be some authorization and cost associated with these types of APIs.
- **Partner APIs** These are only accessible by authorized external developers to aid business-to-business partnerships.
- **Composite APIs** These combine two or more different APIs to address complex system requirements or behaviors.

What is an API endpoint?

- API endpoints are the final touchpoints in the API communication system. These include **server URLs**, services, and other specific digital locations from where information is sent and received between systems.
- API endpoints are critical to enterprises for two main reasons:
 - **1. Security** API endpoints make the system vulnerable to attack. API monitoring is crucial for preventing misuse.
 - **2. Performance** API endpoints, especially high traffic ones, can cause bottlenecks and affect system performance.

REST API security

- Authentication :
 - the process of verifying a user's identification through the acquisition of credentials and using those credentials to confirm the user's identity.
- Authorization
 - the process of allowing authenticated users access to resources by determining whether they have system access permissions. By giving or denying specific licenses to an authenticated user, authorization enables you to control access privileges.
- Hashing the password:
 - With hashing, Users' passwords will stay hidden and secure, even if the database is compromised. hash passwords using a secure one-way hashing algorithm. You can use bcryptjs package
- Security practices: Setting log/sign - in routes, Setting JWT ,Creating Authorization middleware

REST API security (Hashing)

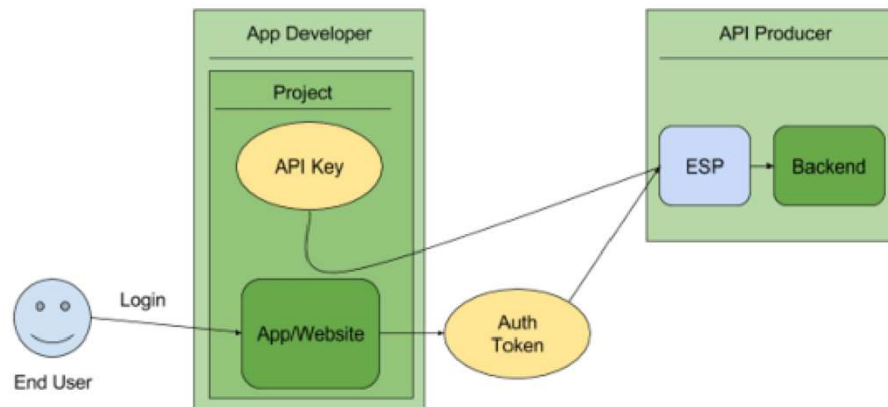
npm i bcryptjs@2.4.3

- let saltRounds=8;
- const hashedPassword = await
 bcrypt.hash(password, saltRounds) ;// The
 hashed password is what would be stored in
 the database
- const isMatch = await
 bcrypt.compare(currentlysentpass,
 hashedPassword);// hashedPassword: the
 retrieved password and currentlysentpass: the
 received password from client login form

```
//hashing identical messages with no salt
hash('mechagodzilla') =
  162e0a91026a28f1f2afa11099d1fcbdd9f2e351095ebb196c90e10290ef1227
hash('mechagodzilla') =
  162e0a91026a28f1f2afa11099d1fcbdd9f2e351095ebb196c90e10290ef1227

//hashing identical messages with random salt
hash('mechagodzilla' + '458cf2979ef27397db67077775225334') =
  f3499a916612e285612b32702114751f557a70606c32b54b92de55153d40d3b6
hash('mechagodzilla' + 'ef5b72eff781b09a0784438af742dd6e') =
  7e29c5c48f44755598dec3549155ad66f1af4671091353be4c4d7694d71dc866
hash('mechagodzilla' + 'cc989b105a1c6a5f0fb460e29dd272f3') =
  6dedd3dbb0639e6e00ca0bf6272c141fb741e24925cb7548491479a1df2c215e
```

Secure a REST API



Authentication tokens

These are used to authorize users to make the API call.

Authentication tokens check that the users are who they claim to be and that they have access rights for that particular API call.

API keys

API keys verify the program or application making the API call.

They identify the application and ensure it has the access rights required to make the particular API call.

API keys are not as secure as tokens but they allow API monitoring in order to gather data on usage.

REST API AUTHENTICATION: Authentication types

1. API Keys Authentication

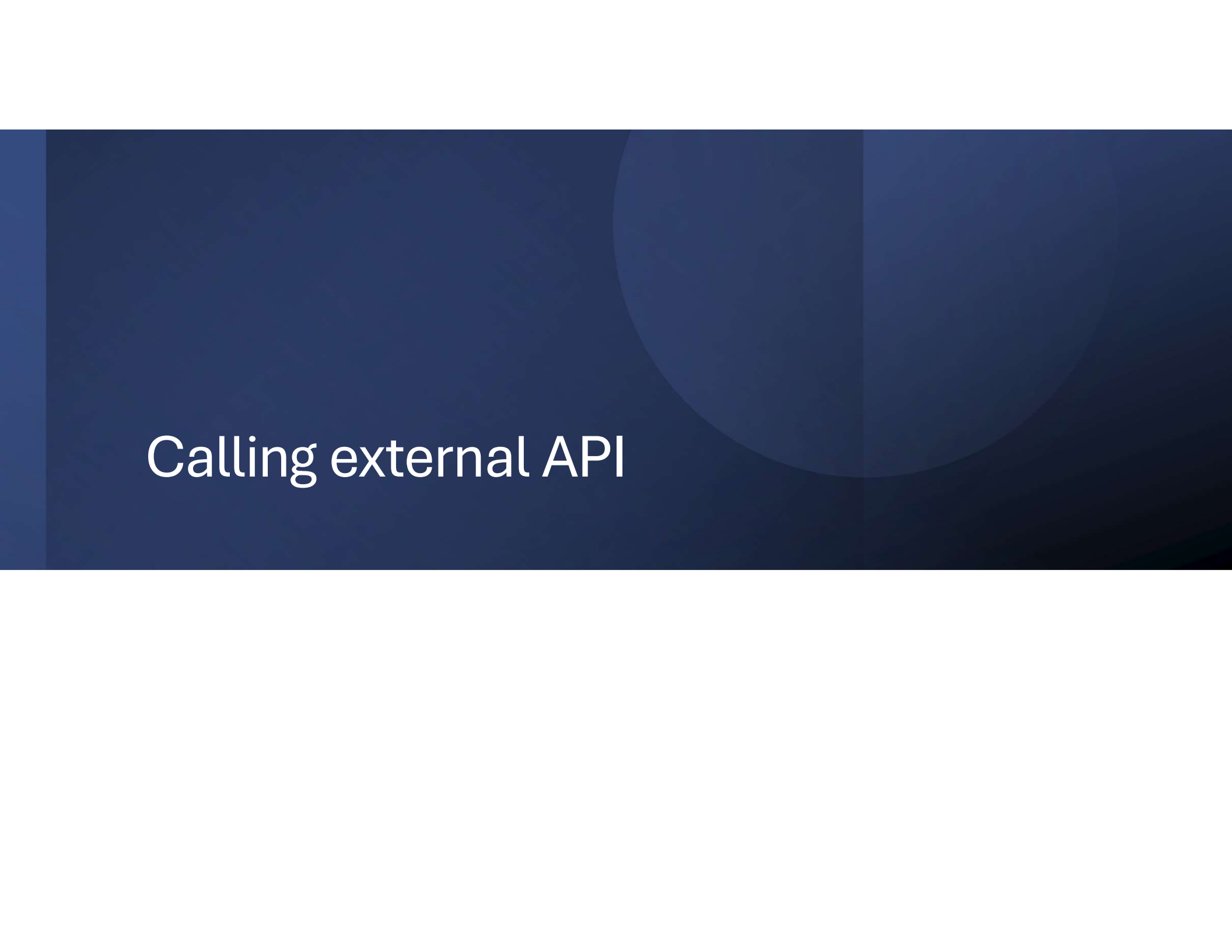
- Clients include a unique static key (e.g., in headers or URLs) to authenticate requests.
- Simple machine-to-machine authentication for APIs.
- Security: Low; keys are long-lived secrets vulnerable to exposure

2. Token based authentication like JWT

Flow:

- User logs in with credentials.
- Server verifies and creates a JWT (with user claims) : by `jwt.sign()`
- sends it or reference to it back according to the transport way
- Client repeatedly send it back with each request
- Server verifies the JWT on each request and grants access if valid and authorized. By `jwt.verify()`

The transport way could be bearer tokens, cookies, session(change into stateful)



Calling external API

Try Free API

- <https://openweathermap.org/current>
- <https://newsapi.org/>

<https://api.openweathermap.org/data/2.5/weather?lat=44.34&lon=10.99&appid={API Key}>

For calling open weather API



Weather in your city

GuideAPIDashboardMarketplacePricingMapsOur InitiativesPartnersBlogFor BusinessSign inSupport centre

Call current weather data

How to make an API call

API call

```
https://api.openweathermap.org/data/2.5/weather?lat={lat}&lon={lon}&appid={API key}
```

Parameters		
lat	required	Latitude. If you need the geocoder to automatic convert city names and zip-codes to geo coordinates and the other way around, please use our Geocoding API
lon	required	Longitude. If you need the geocoder to automatic convert city names and zip-codes to geo coordinates and the other way around, please use our Geocoding API
appid	required	Your unique API key (you can always find it on your account

Product concept

Call current weather data

How to make an API call

API response

JSON format API response example

JSON format API response fields

XML format API response example

XML format API response fields

List of weather condition codes

Min/max temperature in current weather

API and forecast API

Bulk downloading

Other features

Geocoding API

Built-in geocoding

Built-in API request by city name

Built-in API request by city ID

Built-in API request by ZIP code

Format

Units of measurement

Multilingual support

Call back function for JavaScript code

Chat with Ulla - OpenWeather AI assistant

 Ulla Weather Assistant

Express – Weather API Demo

- App.js

```
import express from "express";
import https from "https";
import path from "path";
import { fileURLToPath } from "url";
//Read the current directory name
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const port = 3000;
const hostname = "127.0.0.1";
const appid = 'xxxxxxxxxxxxx';
const app = express();
app.use(express.urlencoded({ extended: true }));
app.use(express.static("public"));

.....
const url =
`https://api.openweathermap.org/data/2.5/weather?lat=${lat}&lon=${lon}&appid=
${API key}`;
```


Express – Weather API Demo

- App.js
cont.

```
app.use(express.static("public"));
app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'index.html'));
});.....
https.get(url, (response) => {
  response.on('data', (data) => {
    const weatherData = JSON.parse(data);
    const temp = weatherData.main.temp;
    const weatherDescription = weatherData.weather[0].description;
    const icon = weatherData.weather[0].icon;
    const iconUrl = `http://openweathermap.org/img/w/${icon}.png`;
    console.log(iconUrl);
    res.write(`<h1>The temperature in ${city} is ${temp} Celcius.</h1>`);
    res.write(`<p>The weather is currently ${weatherDescription}</p>`);
    res.write(``);
    res.send();
  });
  .....
}
```

Streams & Json

Streams

- A stream is a fundamental data structure in Node.js that can read and write data, and send and receive messages, or events. Streaming is implemented in the HTTP module by having classes that are streams.
- In our example, the req and res parameters are streams. Use the on() method to listen to incoming data from a client request like this:

```
req.on('data', (chunk) => {  
  console.log('You received a chunk of data', chunk)  
}))
```

Serve JSON content types

```
res.json({ id: 1, name: "Catcher in the Rye" })
```

```
//The previous Express code method is equivalent to this HTTP module code:  
res.writeHead(200, { 'Content-Type': 'application/json' });  
res.end(JSON.stringify({ id: 1, name: "Catcher in the Rye" })))
```

Create a web app that returns JSON data

```
app.get("/products", (req, res) => {  
  const products = [  
    {  
      id: 1,  
      name: "hammer",  
    },  
    {  
      id: 2,  
      name: "screwdriver",  
    },  
    {  
      id: 3,  
      name: "wrench",  
    },  
  ];  
  
  res.json(products);  
});
```

CRUD API

Creating a CRUD API

It is common to use different HTTP verbs to signal whether we want to create, retrieve, update, or delete (**CRUD**) data.

It is convention to use GET for retrieve requests, POST for create requests, PUT for update requests, and DELETE for delete requests.

For now, our example here will simply modify the in-memory

```
app.get('/companies/:symbol', (req,resp) => {  
  ...  
});  
app.put('/companies/:symbol', (req,resp) => {  
  ...  
});  
app.post('/companies/:symbol', (req,resp) => {  
  ...  
});  
app.delete('/companies/:symbol', (req,resp) => {  
  ...  
});  
;
```

MVC CRUD Example: Authors API

Project Structure

- Project Root
- └─ models/
- | └─ author.js
- └─ routes/
- | └─ authorRoutes.js
- └─ controllers/
- | └─ authorController.js
- └─ views/
- | └─ authors/
- | └─ index.ejs
- | └─ new.ejs
- | └─ edit.ejs
- └─ public/
- └─ app.js
- └─ package.json

Server: app.js

```
const express = require('express');  
const app = express();  
  
app.use(express.urlencoded({ extended: true }));  
app.set('view engine', 'ejs');  
  
const authorRoutes = require('./routes/authorRoutes');  
app.use('/authors', authorRoutes);  
  
app.listen(3000);
```

Routes: authorRoutes.js

```
const express = require('express');
const router = express.Router();
const authorController = require('../controllers/authorController');

router.get('/', authorController.index);
router.post('/', authorController.create);

module.exports = router;
```

Controller: authorController.js

```
const Author = require('../models/author');

index = async (req, res) => {
  const authors = await Author.find();
  res.render('authors/index', { authors });
};

create = async (req, res) => {
  const author = new Author({ name: req.body.name });
  await author.save();
  res.redirect('/authors');
};

module.exports={index,create}
```

Model: models/author.js

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://localhost/authors');
const authorSchema = new mongoose.Schema({
  name: { type: String, required: true }
});
module.exports = mongoose.model('Author', authorSchema);
```

View: authors/index.ejs

```
<h1>Authors</h1>
<ul>
  <% authors.forEach(author => { %>
    <li><%= author.name %></li>
  <% }) %>
</ul>
<a href="/authors/new">Add Author</a>
```

Thank you

